GEMSTAR | TASK 26

CPU Benchmarking, Fusion Quality & Research Validation

An illustrated, source-grounded research report explaining every major Task 26 subtask, the measured findings, their limitations, and how they inform the ARM-FPGA thesis roadmap.

Evidence footprint	Confirmed scope
6,025 measurements	Recorded CPU timings across six workload families
184 comparable groups	Repeated executions grouped by matched workload/input
23 matched GT/EKF frames	Diagnostic position-error analysis only
129 GitHub files	Task 26 source, reports, figures and CSVs committed to main
118 archived file hashes	Final manifest; 114 historical hashes verified before closure

Source: GitHub repository rafiul1232714/GEMSTAR-Reliability-Aware-Adaptive-ARM-FPGA, Task_26_Final_Research, commit a35a1e5. The charts in this PDF are recreated from the committed CSV values; they are not screenshots of the original PNG files.

Prepared: 12 November 2026 | Evidence classification: development-host CPU baseline + diagnostic perception-quality findings.

Executive summary: what Task 26 achieved

Task 26 transformed the preceding GEMSTAR recording and sensor-fusion work into a reproducible development-PC CPU benchmark reference. It measured selected stages independently and a partial sequential LiDAR-Radar replay, checked selected computations against archived outputs, investigated diagnostic ground truth and established statistical variability. It also produced an auditable research archive and a GitHub commit.

Most important takeaway: Task 26 tells us where CPU processing time is spent and where our quality evidence is weak. It does NOT yet tell us how much energy an ARM-FPGA board can save.

Five evidence-backed conclusions

• Computer vision dominates the measured standalone compute cost: YOLO11s on five saved frames had a median-of-frame-medians of 43.746 ms; the detection output was sparse (zero detections in four of five selected frames).

• Camera ROI preprocessing is much cheaper than full-frame preprocessing: for Fog100, median 2.5802 ms (full 640 x 480) versus 0.96025 ms (384 x 288 ROI). This demonstrates lower processing time, not equivalent accuracy.

• LiDAR filtering dominates the partial sequential filter-cluster-radar workload: median of 53 frame medians = 0.7949 ms, excluding Camera, YOLO, association, EKF, transfer, logging and power.

• Radar has low normal batch cost but nontrivial tail variability: median of batch medians 0.008825 ms; an observed maximum 0.617 ms. Batch sizes vary and must not be equated to one return.

• The available EKF position comparison is diagnostic: 23 matched frames; LiDAR-only RMSE 1.663585 m and radar-assisted RMSE 1.640300 m to an approximate GT-center reference. The improvement is only 1.3997%, and independent quality validation remains outstanding.

Why this matters for the research question

The final thesis target is minimum measured total energy per valid perception result, subject to separately justified quality and p99 latency limits. Task 26 supplies the baseline algorithms, timing methods, archived inputs and failure cases needed to design matched comparisons. Later work must measure the ARM processor, FPGA fabric, memory, data transfer and synchronization on the target board.

1 | Task-by-task map (26.1 through 26.8)

Task	What was delivered	What it enables
26.1	CPU development environment and machine inventory.	Hardware/software provenance; reproducible host baseline.
26.2	Benchmark candidate inventory, selected inputs, OSI references, code/input hashes and config.	Comparable test conditions and provenance.
26.3	Original EKF replay benchmark; 100 repeats per workload and MATLAB timing analysis.	Tracking CPU timing and numerical-replay baseline.
26.4	Camera, LiDAR, Radar, YOLO benchmarks; consolidated MATLAB performance figures.	Initial accelerator shortlist and per-stage cost map.

Task	What was delivered	What it enables
26.5	53-frame alignment audit, partial sequential LiDAR-Radar timing, EKF sandbox replay.	Understand actual overlap and computational reproducibility.
26.6	GT source/identity/geometry investigation, diagnostic RMSE, Camera authorization audit.	Quality-gate design and limitations register.
26.7	6,025 sample / 184-group repeatability analysis, Tukey flags, timing plots.	Tail behavior, measurement design and deadline planning.
26.8	SHA-256 manifests, final report, closure and GitHub commit.	Traceability and future CPU-FPGA comparison.

Subtask names here summarize archived outputs and the documented session sequence; they are not claims of end-to-end validation.

2 | Why benchmark a laptop CPU at all?

It is a reference implementation, not the eventual competitor hardware

The recorded host is an AMD Ryzen AI 7 450 (8 cores, 16 threads). A desktop/laptop CPU is convenient for testing correct algorithm outputs, exercising repeatable inputs, profiling each function and detecting failures before complex hardware development. Its results create a software behavior reference and help decide which accelerator designs deserve engineering effort.

The laptop CPU is not a fair direct speedup denominator for an ARM-FPGA hardware claim: different instruction sets, clock frequencies, memory systems, power states and OS behavior make simplistic laptop-versus-FPGA ratios misleading. A defensible hardware experiment needs three levels: (A) x86 host reference for debugging and algorithm discovery; (B) ARM-only reference on the chosen board; (C) matched ARM+FPGA implementation on that same board.

Comparison	Scientifically useful for	Avoid claiming
Laptop CPU vs. itself	ROI/filter variants; algorithm time; reproducibility.	That an FPGA is already faster or lower energy.
Board ARM-only vs. Board ARM+FPGA	Controlled acceleration, transfer overhead and board-level energy.	That faster execution automatically saves joules.
Matched quality in all modes	Whether the optimized path remains a valid perception result.	That lower arithmetic error implies object-detection accuracy.

Fair measurement principle

Measure both candidates using the same input frames, the same declared output requirements, the same validation protocol, and explicit boundaries around input loading, pre/post-processing, inference, association, transfers and synchronization. Record throughput, latency distribution and power over an idle-corrected and board-inclusive measurement window where feasible.

Energy is not execution time. Use measured joules: E = integral of board power over the full measurement interval; report joules / independently validated perception result.

3 | Task 26.3: EKF tracking CPU baseline

The historical EKF benchmark used two different replay workloads: 24 observations (baseline) and 168 observations (seven scenarios), each repeated 100 times. The baseline median replay time was 1.5099 ms, versus 9.46055 ms for the larger seven-scenario replay. These are whole-replay durations; they are not direct per-frame pipeline latency measurements.

EKF replay workload	Median	Descriptive p99	Observations / replay
Baseline	1.5099 ms	2.245492 ms	24
Seven scenarios	9.46055 ms	10.413307 ms	168

The original numerical EKF replay was reproduced in a protected sandbox for all 24 archived records with zero mismatches. This supports computational reproducibility and permits checking that later CPU/FPGA implementations match the intended algorithm. It does not verify the ground-truth target identity, physical accuracy, or use of an authorized Camera measurement.

Later use

In Task 27, record EKF as a separately timed ARM-side software component. In Tasks 31-36, only consider EKF hardware offload if an ARM-board benchmark shows that latency/energy improvements justify the numeric-verification complexity. In Tasks 37-41, retain the same EKF output checks when switching scheduling modes.

4 | Task 26.4: Camera preprocessing and YOLO inference

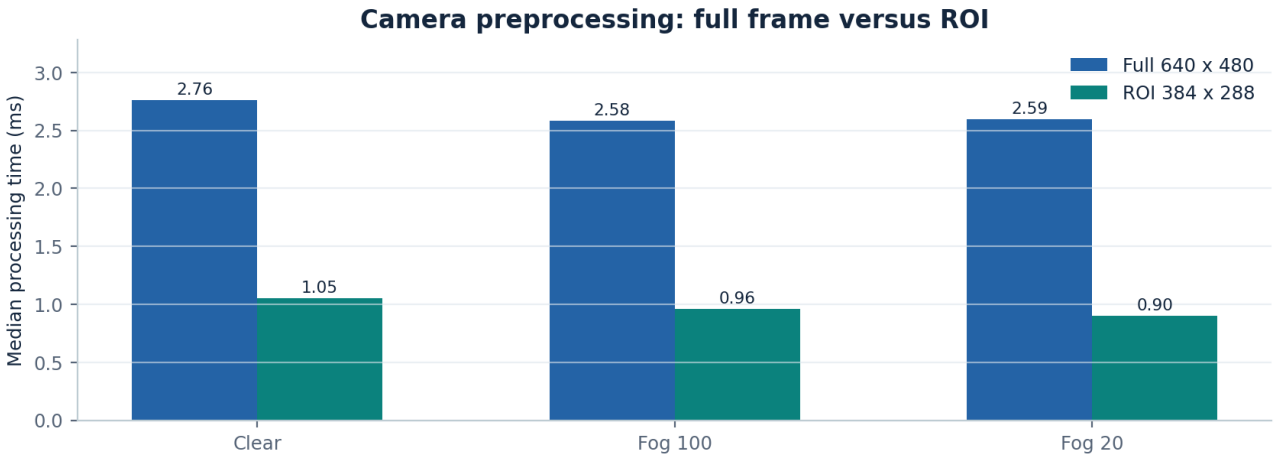


Figure 1. Archived Camera preprocessing medians for each image/weather variant; 100 timed repetitions per weather/mode group. Source: task26_4B_camera_cpu_summary.csv.

Each full image contains 307,200 pixels; the ROI contains 110,592 pixels (36% of the full frame). The Fog100 case showed a fall from 2.5802 ms to 0.96025 ms. Similar runtime differences occurred in clear and dense-fog conditions. Reducing pixels reduces measured CPU work, but it may discard objects outside the ROI. Any future use of ROI scheduling must measure missed detections and spatial coverage.

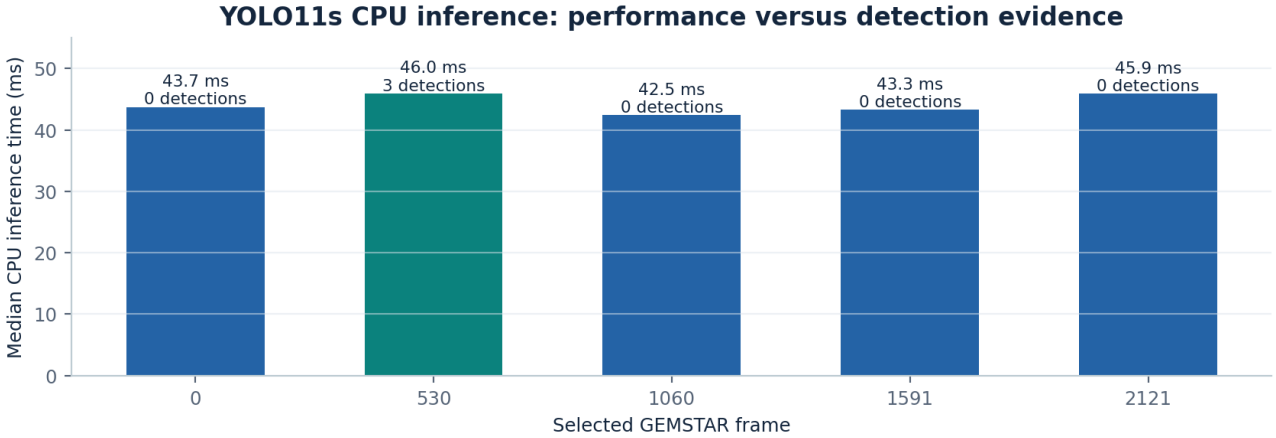


Figure 2. YOLO11s inference medians and detections on five stored Camera frames. The model produced 3 detections only on frame 530; results do not establish recall or classification quality. Source: task26_4J_yolo_cpu_summary.csv.

YOLO11s CPU inference used 3 warmups and 10 timed executions per selected image, for 50 measurements. The median of five frame medians is 43.74605 ms; its highest descriptive group p99 is 51.676415 ms. The small, nonrepresentative set is insufficient for hard p99 or perception-quality claims. AI acceleration is only compelling if inference produces useful validated outputs and the board supports model operators and quantization without unacceptable accuracy loss.

Later use

Task 27: compare full/ROI Camera configurations on fixed scenes and evaluate a better calibrated detection model. Tasks 28-29: decide whether conventional image-processing FPGA stages or INT8 Camera neural inference justify implementation. Tasks 31-36: test actual hardware throughput, memory traffic, model accuracy and joules per valid detection.

5 | Task 26.4: LiDAR and Radar CPU stages

LiDAR replay used 53 selected frames, 25 repeats each for filtering and clustering (2,650 timed stage runs). The tested source was extracted from the original Task 24 implementation, and archived cluster counts were reproduced for all 53 frames. Filtering usually consumes more time than clustering in the observed frames, but clustering cost depends on the number of surviving points, clusters and frame geometry.

Radar validation used 12 batches over selected paired inputs, each repeated 100 times (1,200 timed batch runs). The input batches had different return counts. The median of group medians was 0.008825 ms; observed maximum 0.617 ms. Interpret the latter as an outlier worth investigating, not as proven recurrent worst-case behavior.

What is probably hardware-friendly?

The LiDAR filter and coordinate/geometric tests are candidates for streaming pipelined FPGA logic because their operations are regular and can be checked numerically against the existing CPU code. Clustering is a secondary candidate whose parallelism and memory structure require profiling. Radar validation is cheap in absolute CPU time, so moving it across a CPU-FPGA interconnect could cost more energy than it saves; that question remains experimental.

6 | Task 26.5: Partial integrated LiDAR-Radar pipeline

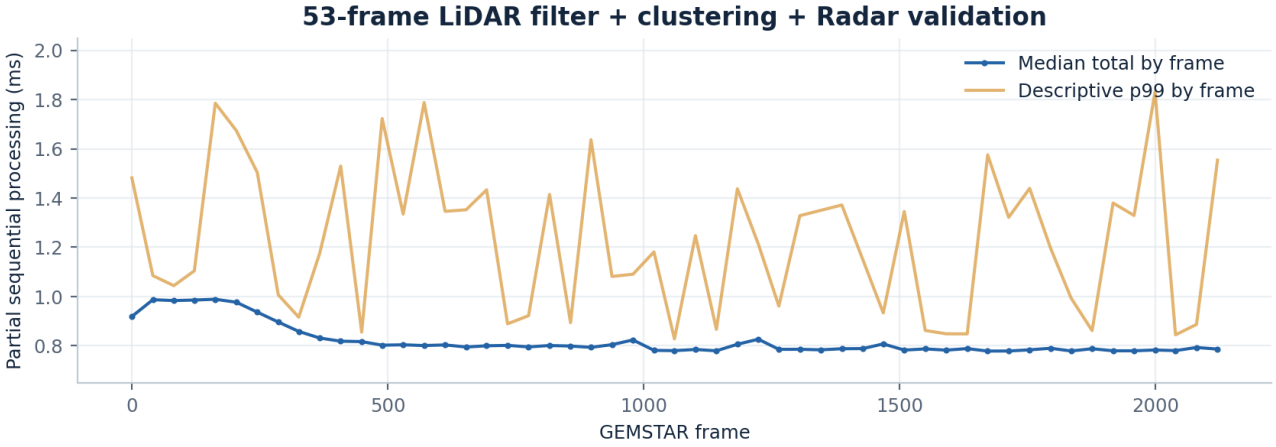


Figure 3. Per-frame medians and descriptive p99 values, from 25 repeats for each of 53 frames. The total is sequential LiDAR filtering + clustering + Radar validation only. Source: task26_5C_integrated_cpu_summary.csv.

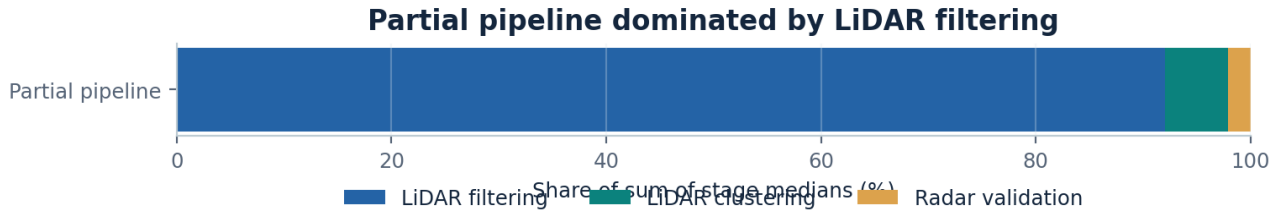


Figure 4. Relative stage cost based on medians across frame groups; descriptive view only. Actual per-frame stage shares differ. Source: task26_5C_integrated_cpu_summary.csv.

The median of per-frame total medians was 0.7949 ms, whereas the highest per-frame descriptive p99 was 1.831704 ms and the maximum individual measured execution was 2.0275 ms. Those numbers are useful for identifying processing-stage sensitivity; they do not include Camera, YOLO, actual object association, EKF, CSV/OSI decoding, data movement, logging, dashboard updates or board energy.

Input alignment was also checked: LiDAR point/cloud and Radar data covered 53 frames, Camera provided five decoded images, and the archived EKF evaluation included 24 frames. Only frames 0 and 530 overlapped across all four input sources. This constrains what can be called a matched three-sensor validation experiment.

Later use

Use this partial pipeline as a software integration smoke test and arithmetic reference. Do not extrapolate its 0.79 ms median into live end-to-end GEMSTAR throughput. Tasks 42-46 must include streaming buffers, OSI decoding/ROS2 transport, synchronization, scheduler decisions and data-transfer costs within an explicitly measured interval.

7 | Task 26.6: Fusion quality and ground-truth diagnostics

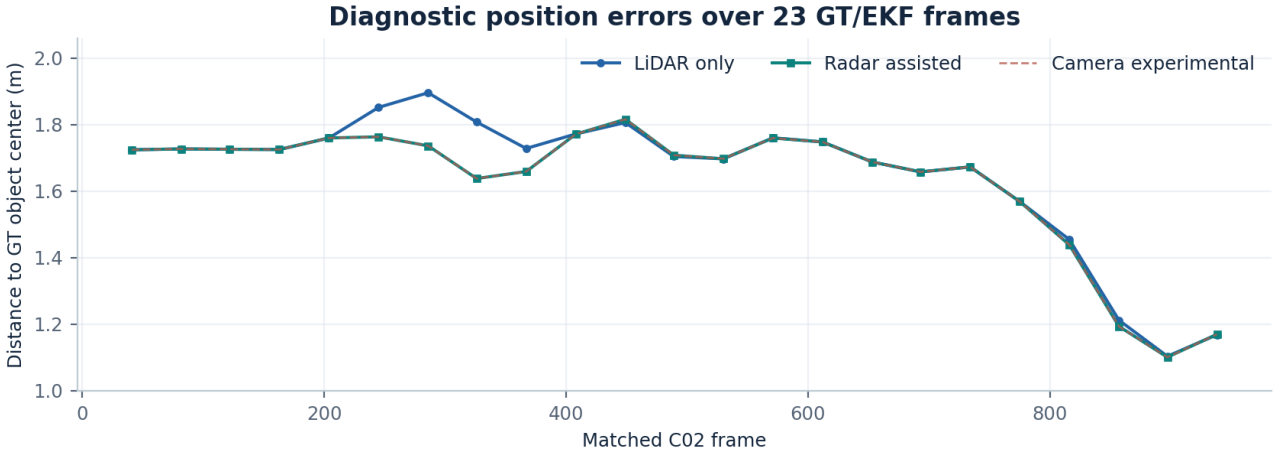


Figure 5. Observed position error to GT object center over 23 matching C02 frames. Camera was experimental (only the frame-530 update was accepted); plotted Camera differences from Radar are extremely small. Source: task26_6C_ekf_position_errors_by_frame.csv.

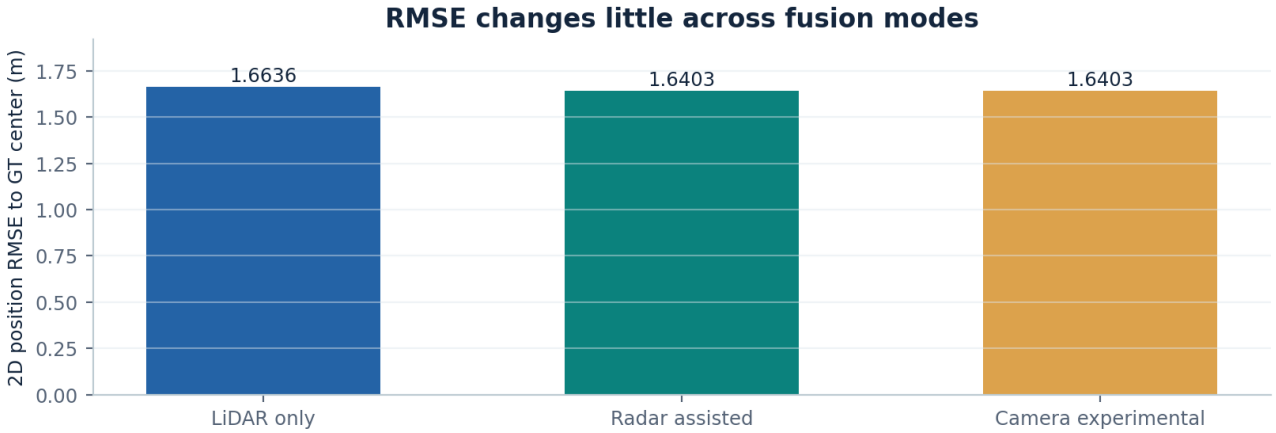


Figure 6. Diagnostic 2D position RMSE computed from 23 matching frames. These values are not independently validated final tracking accuracy. Source: task26_6C_ekf_position_error_summary.csv.

The quality audit found 23 common GT/EKF timestamps (all matched), target object ID 76897 and 69 internal error-consistency checks. The diagnostic GT-center 2D RMSE values were 1.663585 m (LiDAR only), 1.640300 m (Radar assisted) and 1.640308 m (Camera experimental). The Radar-versus-LiDAR difference equals 0.023285 m, approximately 1.3997% on this one matched sequence. A tiny aggregate improvement is not evidence of robust multi-weather fusion advantage.

A crucial audit found Camera update authorization = NO despite one experimental accepted frame (530). The code could be replayed numerically, but this update cannot support an authorized Camera-fusion accuracy claim. The ground-truth transform was explicitly marked PRELIMINARY_NOT_VALIDATED, and target identity was an evaluation-only association. Actor velocity fields also had six zero-velocity-during-motion flags.

Geometry: why an apparent 1.6 m error may be misleading

The mean Radar-assisted signed X bias was -1.601283 m, with mean Y bias -0.116574 m. The Radar mean distance to the GT center was 1.628205 m; to an approximate rear-face location it was 0.639567 m. These are two distinct reference points: 0.639567 m is NOT a corrected RMSE and must not replace the center RMSE.

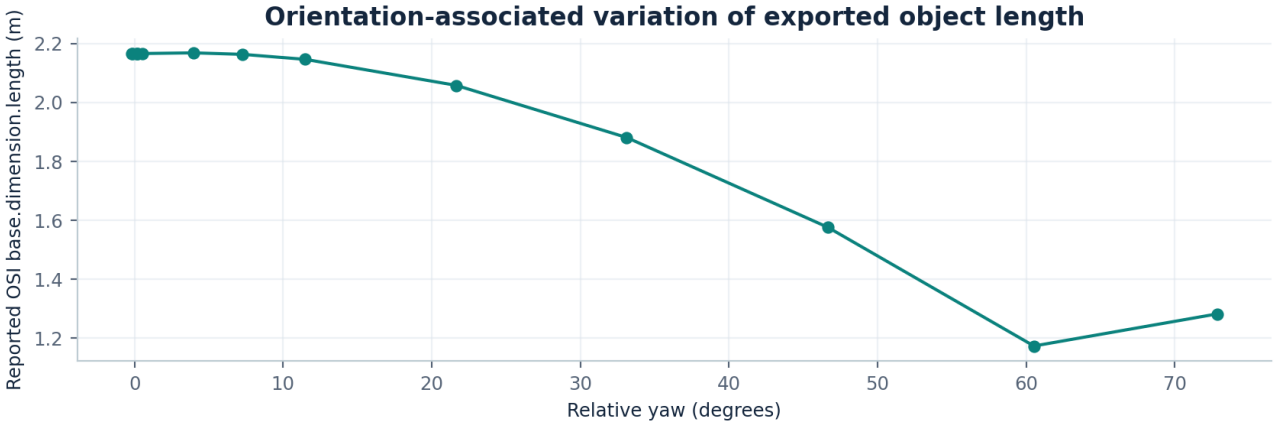


Figure 7. Orientation-associated changes in the OSI-exported target length (sampled from archived Task 26 findings). The original source reads base.dimension.length directly; why it varies is not established.

The constructed rear-face coordinate matches the formula center minus half the exported target length along target heading in all 23 frames, with numerical residual zero. But the reported OSI length varied from 1.1733 to 2.1687 m as heading changed. Without verification of exporter semantics, physical vehicle geometry and actual LiDAR-visible surface, no geometric offset should be fitted or removed.

What quality must be proven next?

Task 27 must define matched identity checks, missed/false detections, calibrated Camera updates, comparable object-center or surface-based position metrics, reference-velocity integrity and quality acceptance conditions before optimizing speed or energy. Numeric quality thresholds and p99 deadlines were not set by Task 26; they require justified requirements and independent evaluation data.

8 | Task 26.7: Repeatability, p99 and outliers

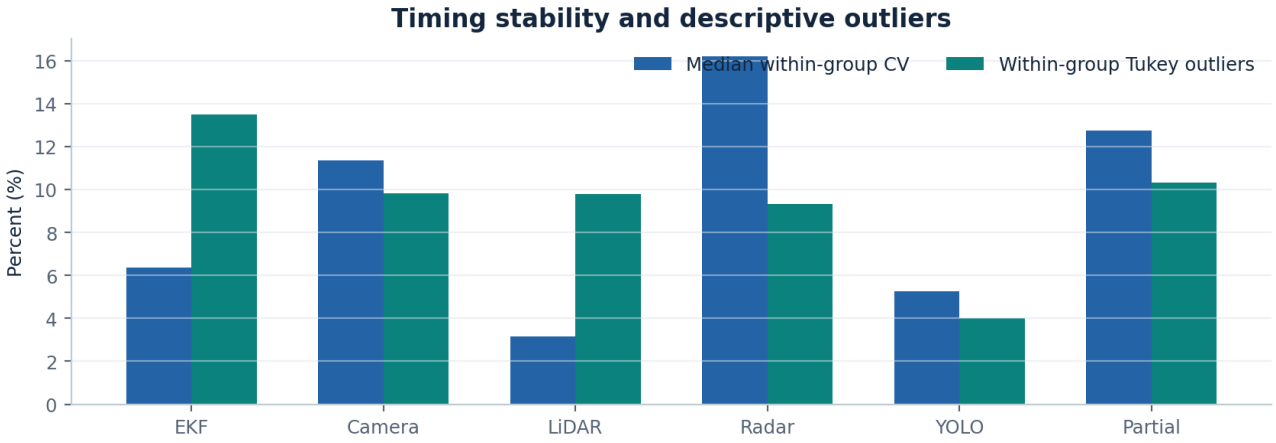


Figure 8. Median within-group timing coefficient of variation (CV) and Tukey outlier rates for six workload families. Outliers were retained, not deleted. Source: task26_7B_repeatability_by_workload.csv.

Workload	Runs	Groups	Median group CV	Max observed
EKF	200	2	6.379%	10.8892 ms
Camera	600	6	11.371%	5.1668 ms
LiDAR	2,650	106	3.163%	1.7359 ms
Radar	1,200	12	16.222%	0.6170 ms

Workload	Runs	Groups	Median group CV	Max observed
YOLO11s	50	5	5.262%	51.8381 ms
Partial pipeline	1,325	53	12.735%	2.0275 ms

Method: per-group means/medians, sample standard deviations, CV, linearly interpolated descriptive p95/p99 and Tukey 1.5 x interquartile range flags. The 184 group results should not be flattened into a single common latency percentile. In particular, only 10 YOLO repeats and 25 LiDAR/integrated repeats per group are insufficient to certify p99 or worst-case execution time.

The Radar pooled median of 0.0108 ms differs from its median-of-batch-medians of 0.008825 ms. The difference is a grouping/aggregation issue, not a contradiction. Radar normal cost is tiny, but one 0.617 ms observation is approximately 57.1 times the pooled median and merits controlled repeat testing on the real platform.

9 | Task 26.8: Reproducibility, archive and GitHub

The research archive was organized into code, CSV, configurations, plots, raw-data references, research findings and a research-report folder. Twenty required outputs were present. The original 114-file audit produced SHA-256 hashes; all 114 historical hashes were rechecked with zero mismatches. A final four-folder manifest recorded 118 files and later generated verification files were preserved.

The safe GitHub upload committed 129 research files to main at commit a35a1e5895a5466ac970321ffa89cf0f16ffd7c8. The push verification showed identical local and remote commit hashes. A Python virtual environment, raw OSI binaries and other unnecessary machine-specific files were excluded. Raw OSI manifests help trace recordings but are not themselves complete raw-data replication.

What to cite/reuse from this archive

Evidence need	Preferred archived reference
CPU profiling	02_CSV_Data/task26_4L_consolidated_cpu_summary.csv
Per-frame partial pipeline	02_CSV_Data/task26_5C_integrated_cpu_summary.csv
Repeatability / outliers	02_CSV_Data/task26_7B_repeatability_by_group.csv
GT tracking quality	02_CSV_Data/task26_6C_ekf_position_errors_by_frame.csv
Limitations / criteria	06_Research_Findings/task26_6E_quality_constraints_and_limitations.txt
Main narrative	06_Research_Findings/task26_8B_final_cpu_research_report.md
Provenance	02_CSV_Data/task26_8C_final_archive_manifest.csv

10 | How Task 26 informs the next roadmap

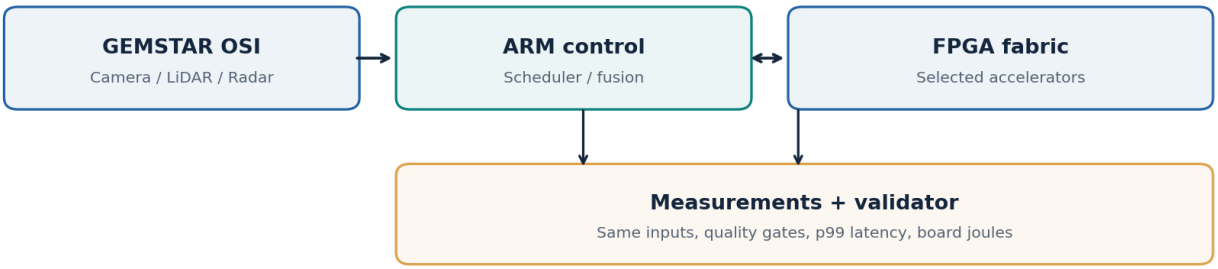


Figure 9. Candidate future processing structure, drawn as a design concept rather than an implemented or benchmarked ARM-FPGA system.

Future stage	How Task 26 helps	What evidence is still needed
27: Software design space	Choose candidate ROI, filter, batching, detection and fusion configurations based on measured cost.	Matched quality results and exact comparable processing boundaries.
28-29: Partition selection	Rank streaming LiDAR stages and Camera AI candidates; establish software fallback.	Board feasibility, FPGA resources, memory traffic and fair ARM-only baseline.
30: Board bring-up	Reuse input manifests and CPU reference outputs for correctness tests.	Real board setup, driver/AXI interfaces, synchronization, measurement access.
31-36: FPGA acceleration	Compare FPGA output against original CPU code on identical inputs.	FPGA utilization, actual latency/p99, board watts and joules.
37-41: Adaptive scheduler	Use workload costs and known failure cases to design policies.	Per-scene reliability, calibrated risk limits, power-vs-quality tradeoff.
42-46: Live demo	Reuse C02 alignment and stage-timing lessons.	Continuous GEMSTAR stream, end-to-end timestamping, dashboard measurements.
47-50: Thesis validation	Provide frozen baseline and an auditable comparison trail.	Independent scenarios, confidence intervals, quality gates, energy claims.

Probable architecture, not yet selected

Camera: traditional FPGA preprocessing with an optional quantized AI inference accelerator, conditional on real detection quality and board support. LiDAR: conventional streaming/geometric FPGA candidate, with possible clustering acceleration. Radar: lightweight ARM checks unless stronger evidence justifies FPGA offload. EKF/association: ARM-side verified reference first. Scheduler: ARM decisions using validated scene reliability, deadlines and board energy. A full LiDAR AI detector is optional, not automatically needed.

11 | Proposed comparison and validation protocol

1. Freeze immutable OSI input pairs and scenario IDs (clear, fog, traffic), plus target/camera calibration and independent labels.

2. Define exactly what one valid perception result is: target identity, detection and tracking metric, allowed misses, and acceptable reference geometry.

3. Predeclare objective thresholds (e.g., quality floor and p99 deadline) based on requirements; do not tune them after seeing a fast candidate.

4. Benchmark ARM-only and ARM+FPGA on the same target board using identical inputs and matching software boundaries, including transfer/synchronization.

5. Collect many independent scenes and repeated runs; report per-scene p95/p99, maxima, outliers and confidence intervals without deleting slow samples.

6. Measure board rails or suitably instrumented full-system power under defined idle and active conditions. Integrate joules over an explicit interval.

7. Report joules per valid output, total-valid-output count, frames dropped, accuracy/recall, resource use, and reliability under fog and traffic.

8. Accept a scheduler or accelerator only if its independently verified quality floor and latency constraints pass; then compare energy against matched baselines.

Measurements that must never be misrepresented

Available now	Not supported yet
Laptop CPU stage medians	Board ARM-only speedup or FPGA energy savings
Partial LiDAR+Radar sequential median	Live all-sensor end-to-end latency
23-frame GT-center diagnostic RMSE	Validated detection/track accuracy across all conditions
Descriptive group p99 values	Certified worst-case latency or real-time deadline satisfaction
Approximate GT rear-face distance	True corrected object-center RMSE or independent surface localization

12 | Research conclusions and questions to answer

Task 26 is a successful CPU benchmarking and audit milestone, not a finished energy-saving result. Its highest scientific value is a reproducible, quality-aware baseline that prevents later FPGA comparisons from becoming misleading.

What appears promising

Camera ROI preprocessing offers substantial observed processing-time reduction with a potential field-of-view tradeoff. YOLO11s is a large CPU workload but cannot be called an acceleration success until detection quality is validated. LiDAR filtering is a plausible hardware pipeline candidate. Radar is cheap enough that offload overhead may dominate. EKF replay is computationally reproducible and useful as an ARM reference.

What could invalidate a future claim?

Comparing a laptop to a different ARM+FPGA board without an ARM-only control; treating one Radar time outlier as a universal worst case; assuming a few images prove CNN accuracy; interpreting all 53 LiDAR/Radar frames as full three-sensor overlap; subtracting a rear-face offset from GT-center RMSE; declaring a p99 deadline from groups of ten runs; or claiming energy savings from milliseconds alone.

Immediate Task 27 experiments

Begin with a decision matrix pairing each candidate configuration with its measured CPU cost, independent quality requirements, candidate ARM/FPGA location, expected data-transfer overhead, memory footprint, and

uncertainty. Run software-only matched tests first. Choose Task 29 partition candidates that maximize expected verified joules-per-valid-result improvement, not only peak throughput.

Source and provenance notes

GitHub:
https://github.com/rafiul1232714/GEMSTAR-Reliability-Aware-Adaptive-ARM-FPGA/tree/main/tests/Task_26_Final_Research

Commit: a35a1e5895a5466ac970321ffa89cf0f16ffd7c8

Primary source files used: task26_8B_final_cpu_research_report.md; task26_4B_camera_cpu_summary.csv; task26_4J_yolo_cpu_summary.csv; task26_5C_integrated_cpu_summary.csv; task26_6C_ekf_position_errors_by_frame.csv; task26_7B_repeatability_by_workload.csv; task26_4D_lidar_cpu_summary.csv; task26_4G_radar_cpu_summary.csv. Where findings are not present in these CSV files, they are attributed to the archived final narrative and documented audits.

Chart provenance: Figures 1, 2, 3, 5, 6, 8 are reconstructed from source CSV values; Figure 4 uses group-level stage medians as a descriptive visualization; Figure 7 uses reported selected GT geometry points from archived subtask outputs; Figure 9 is a proposal. Original MATLAB-generated PNGs exist on GitHub but are not embedded as binary originals in this PDF.

Evidence classification: observed = archived numbers; calculation = arithmetic derived from archived values; interpretation = hypothesis requiring later validation. No external literature used to assert new hardware-performance results.